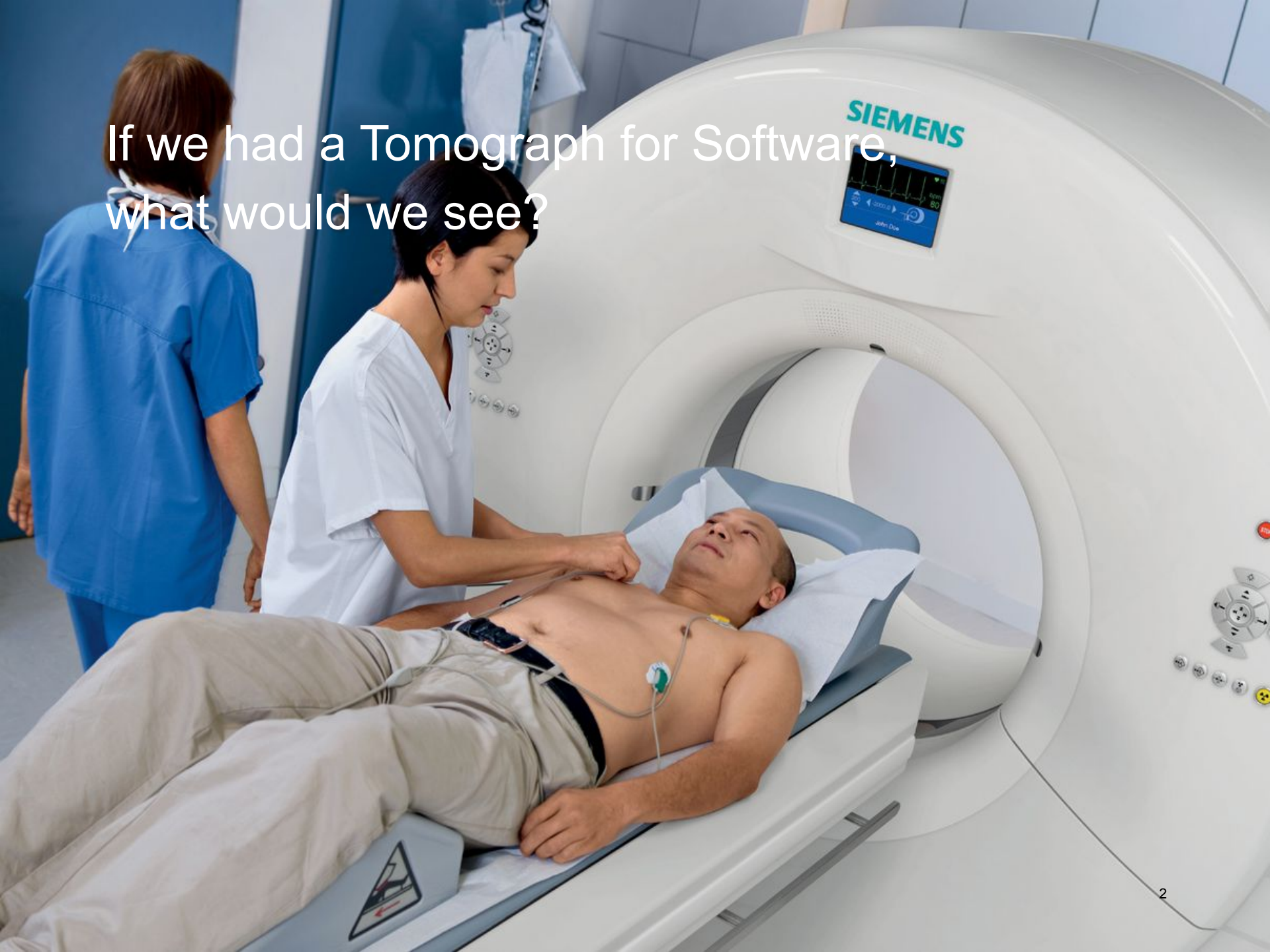
Hochschule Konstanz
Fakultät Informatik

Software Engineering

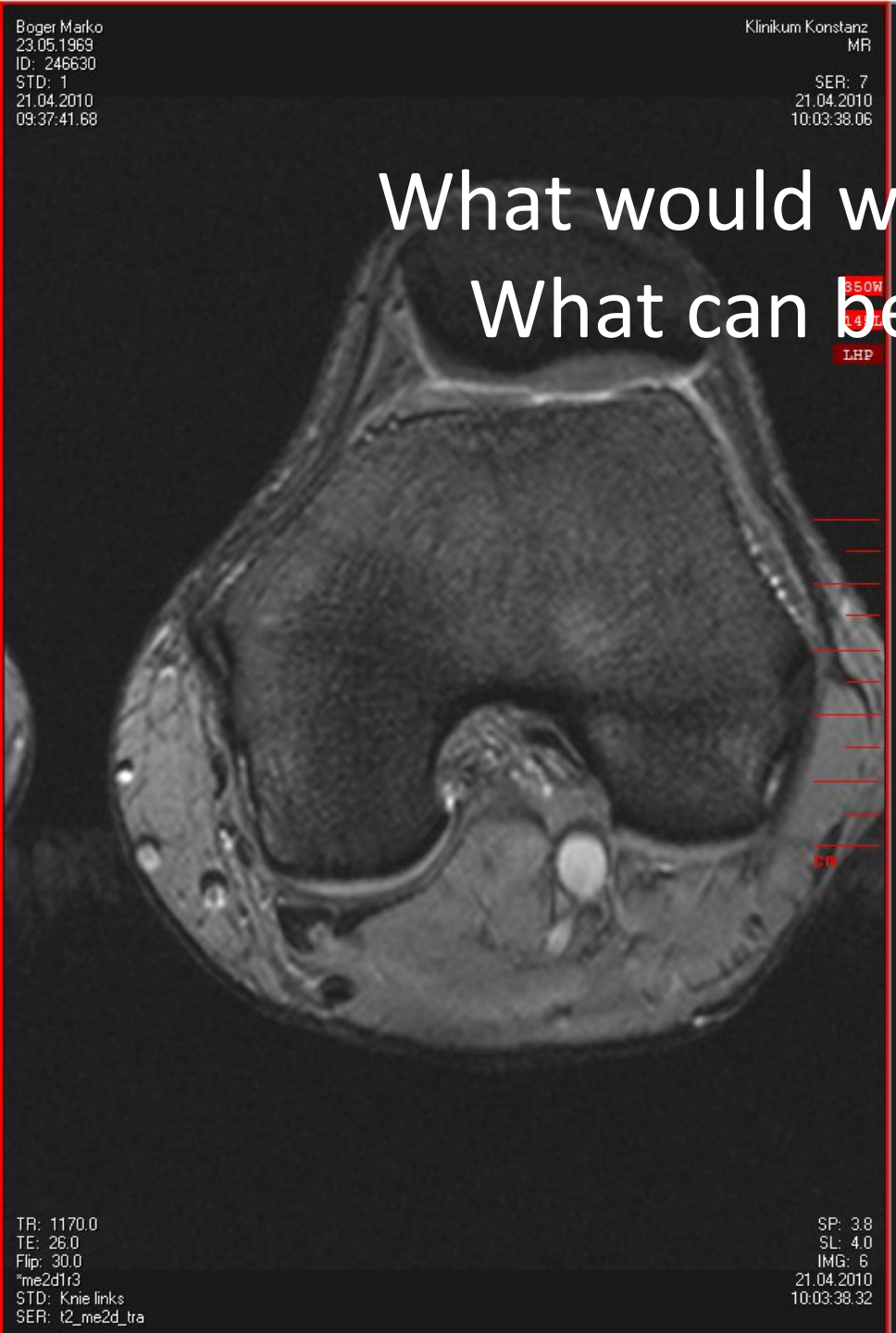
05 - Architecture

Prof. Dr. Marko Boger



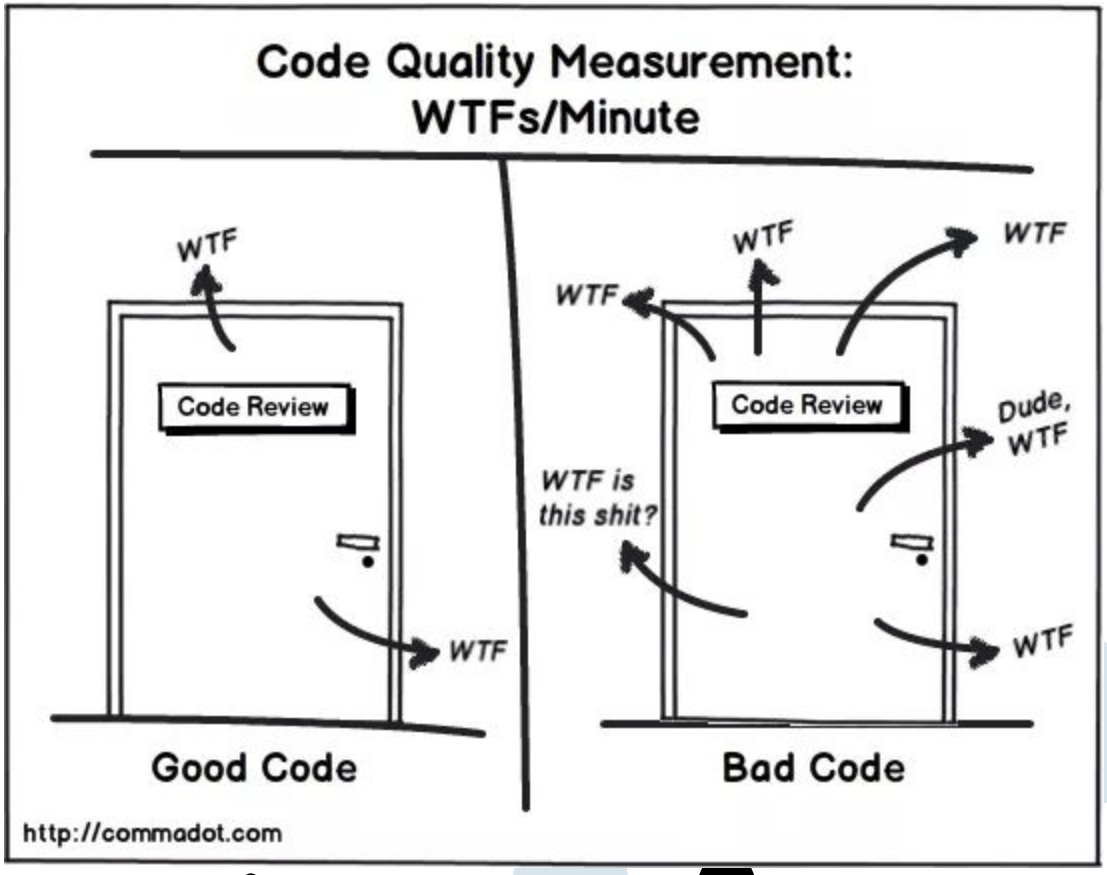
If we had a Tomograph for Software, what would we see?

What would we want to see?
What can be visualized?



Manual Code Review

Manual code reviews are very time consuming and imprecise.



Static Code Analysis

Static code analysis is performed without actually executing the code. Tooling looks for patterns in code that often lead to bugs and warns about it.

Examples are

- Findbugs
- PMD
- SonarQube

Tools collect metrics about the code, like

- Cyclomatic complexity
- Technical dept

Cyclomatic Complexity

Counts the number of if-branches and for-loops

Cyclomatic complexity is one of the most important metrics for imperative code.

Cyclomatic complexity becomes meaningless in FP.

Sample code

```
statement 1;
If ( condition 1 ) {
        statement 2;
} else {
        statement 3;
}
statement 4;
for( condition 2 ) {
        statement 5;
}
statement 6;
```

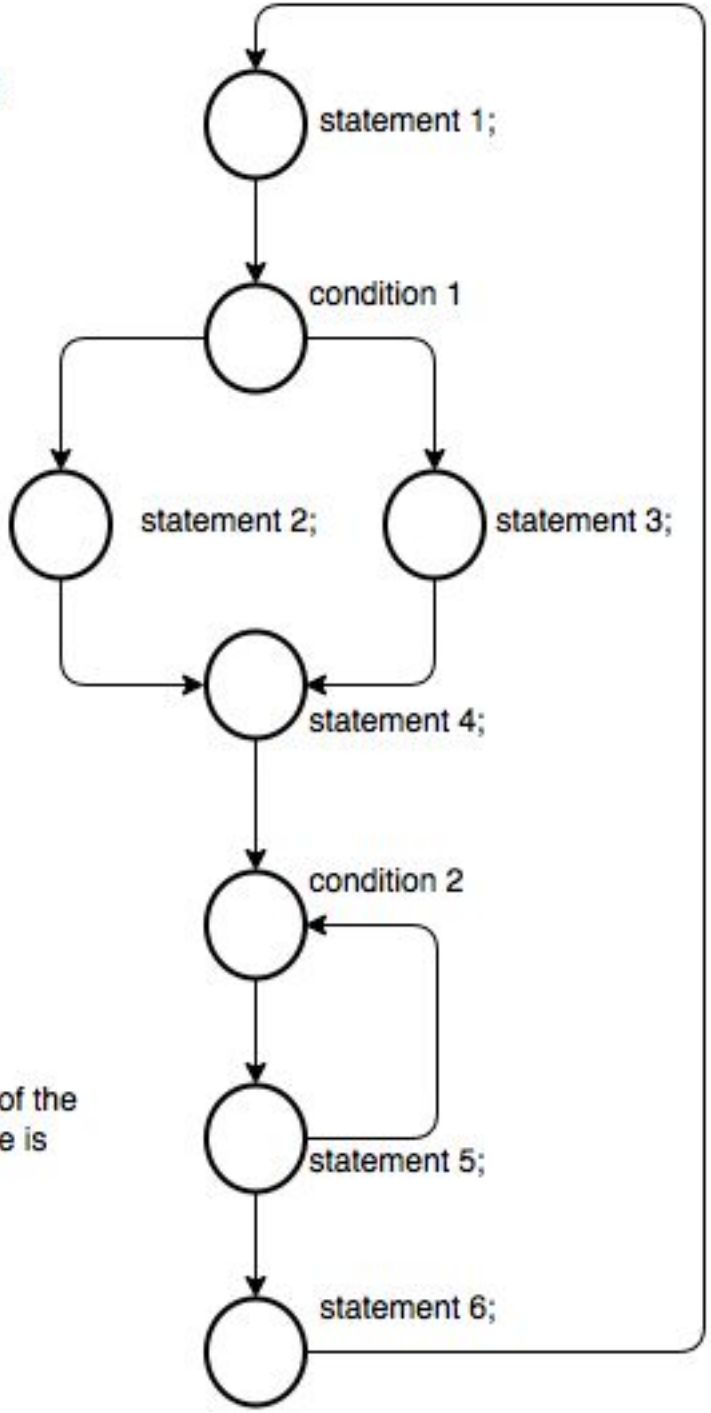
Complexity

The cyclomatic complexity of the graph representing the code is

$v(G) = E - N + 2$

$= 9 - 8 + 2$

$= 3$



Technical Debt

Technical debt is an aggregated metric for code quality.

It sums up the time necessary to fix the problems from static code analysis, based on heuristics.

It can be expressed in hours, money or as per-cent. In per-cent it is the relation of time needed to invest to clean up the code to time already invested in the code.

Technical debt in percent can be compared to an interest rate on a bank loan.

SonarQube

B bt-openlink / bt-openlink master

September 20, 2018, 6:50 PM Version 0.0.11-SNAPSHOT

Overview Issues Security Reports Measures Code Activity

Quality Gate Failed

9.5% Duplicated Lines on New Code (%) is greater than 3.0%

C Reliability Rating on New Code is worse than A

Bugs Vulnerabilities

New code: last 30 days

7 C Bugs

0 A Vulnerabilities

7 C New Bugs

0 A New Vulnerabilities

Code Smells

4d A Debt

started last year

81 Code Smells

1d A New Debt

28 New Code Smells

Coverage

89.8% Coverage

481 Unit Tests

93.5% Coverage on 854 New Lines to Cover

Duplications

10.3% Duplications

105 Duplicated Blocks

9.5% Duplications on 4.9k New Lines

About This Project

No tags

M 12k Lines of Code

Java 12k

Project Activity

September 20, 2018
0.0.11-SNAPSHOT

September 20, 2018
Project Analyzed

September 20, 2018
Project Analyzed

Show More

Quality Gate

(Default) Sonar way

Quality Profiles

(Java) Sonar way

Project Key

com.bt.openlink:bt-openlink Copy

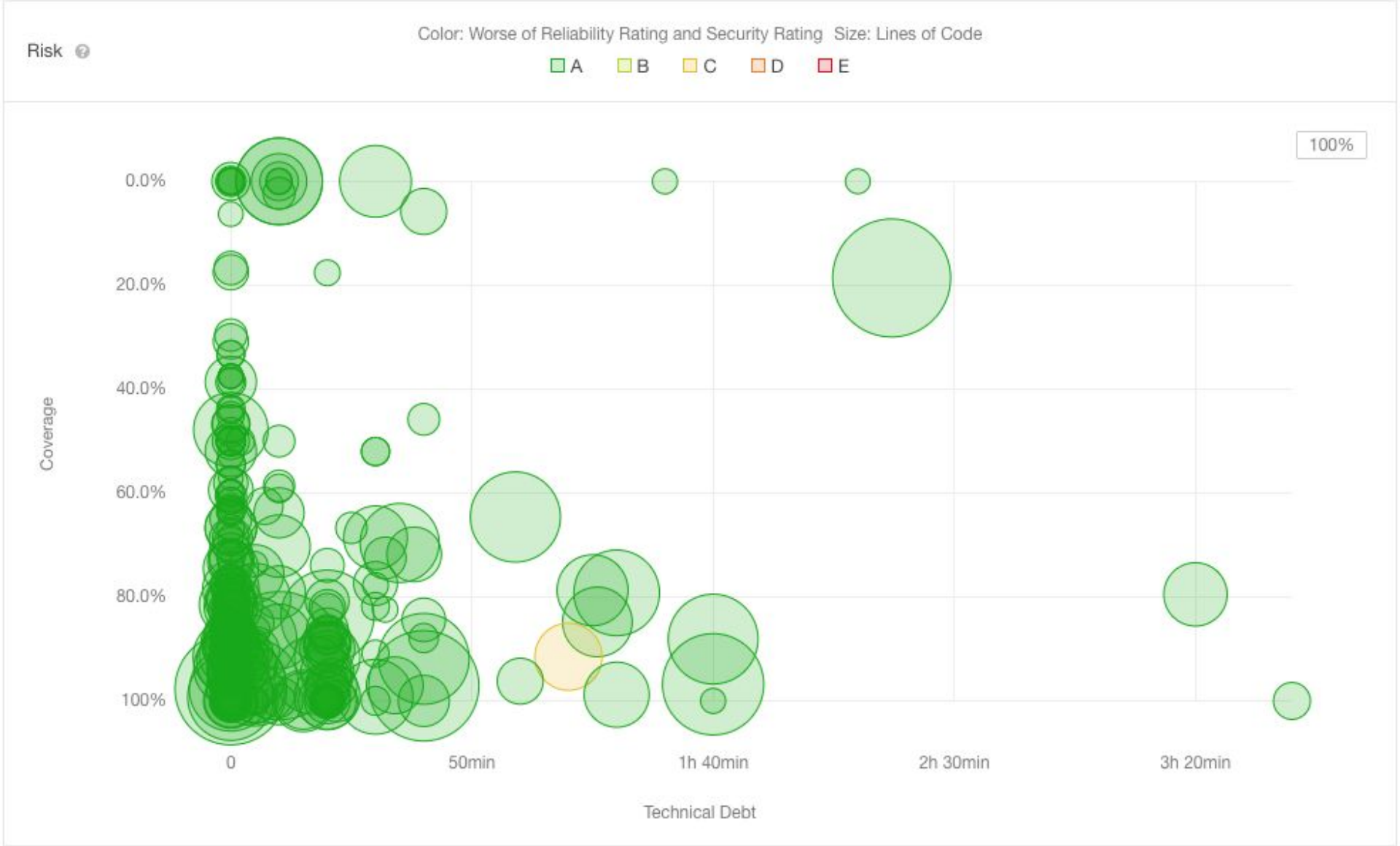
Organization Key

bt-openlink Copy

Get project badges

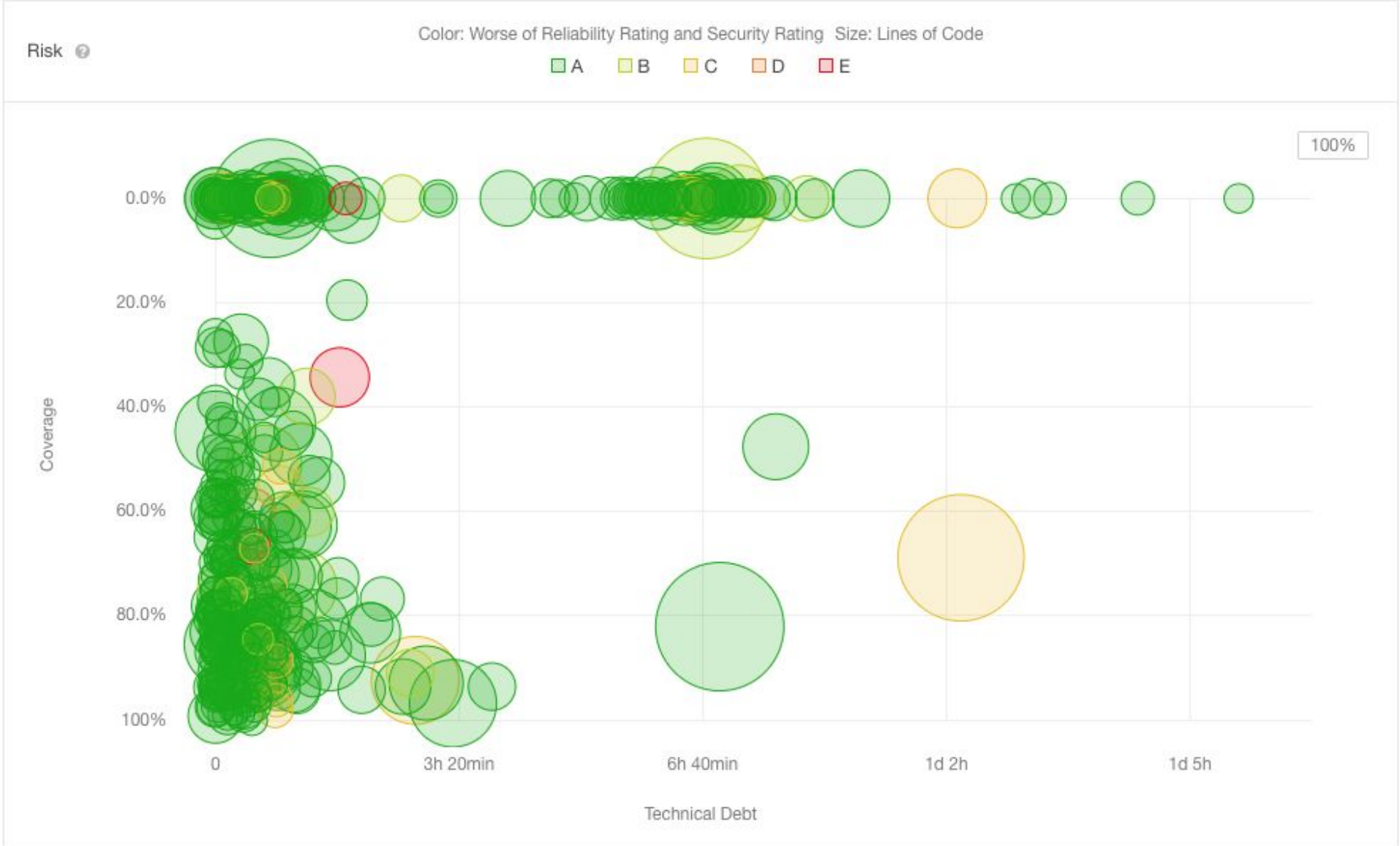
SonarQube

Hochschule Konstanz
Fakultät Informatik



SonarQube

Hochschule Konstanz
Fakultät Informatik



Structural Analysis

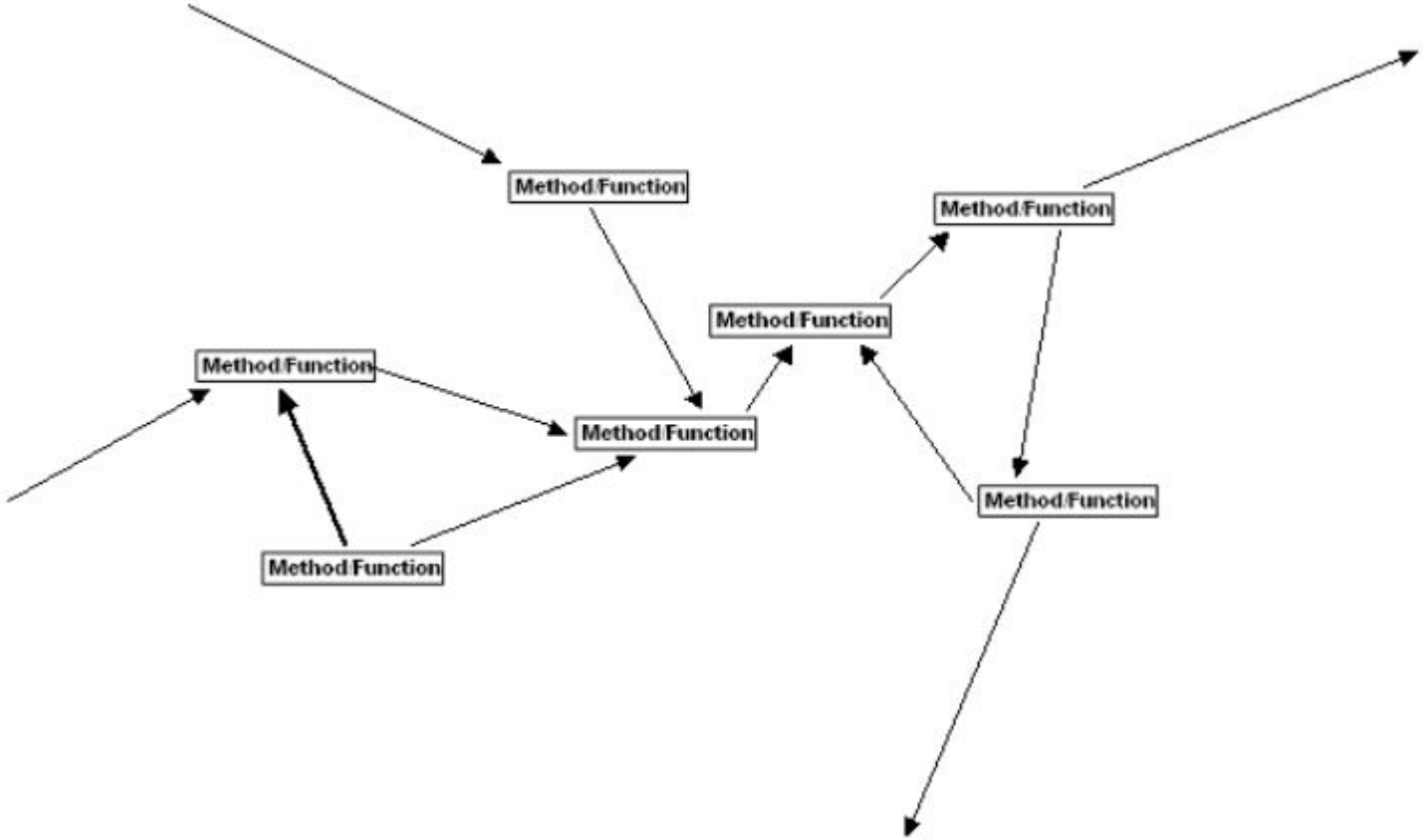
Instead of collecting metrics, we want to look at the architecture of software. For that we want to analyse the dependencies of code fragments.

There are several ways to visualize such dependencies, depending on the granularity of the fragments.

Analysis of Method Calls

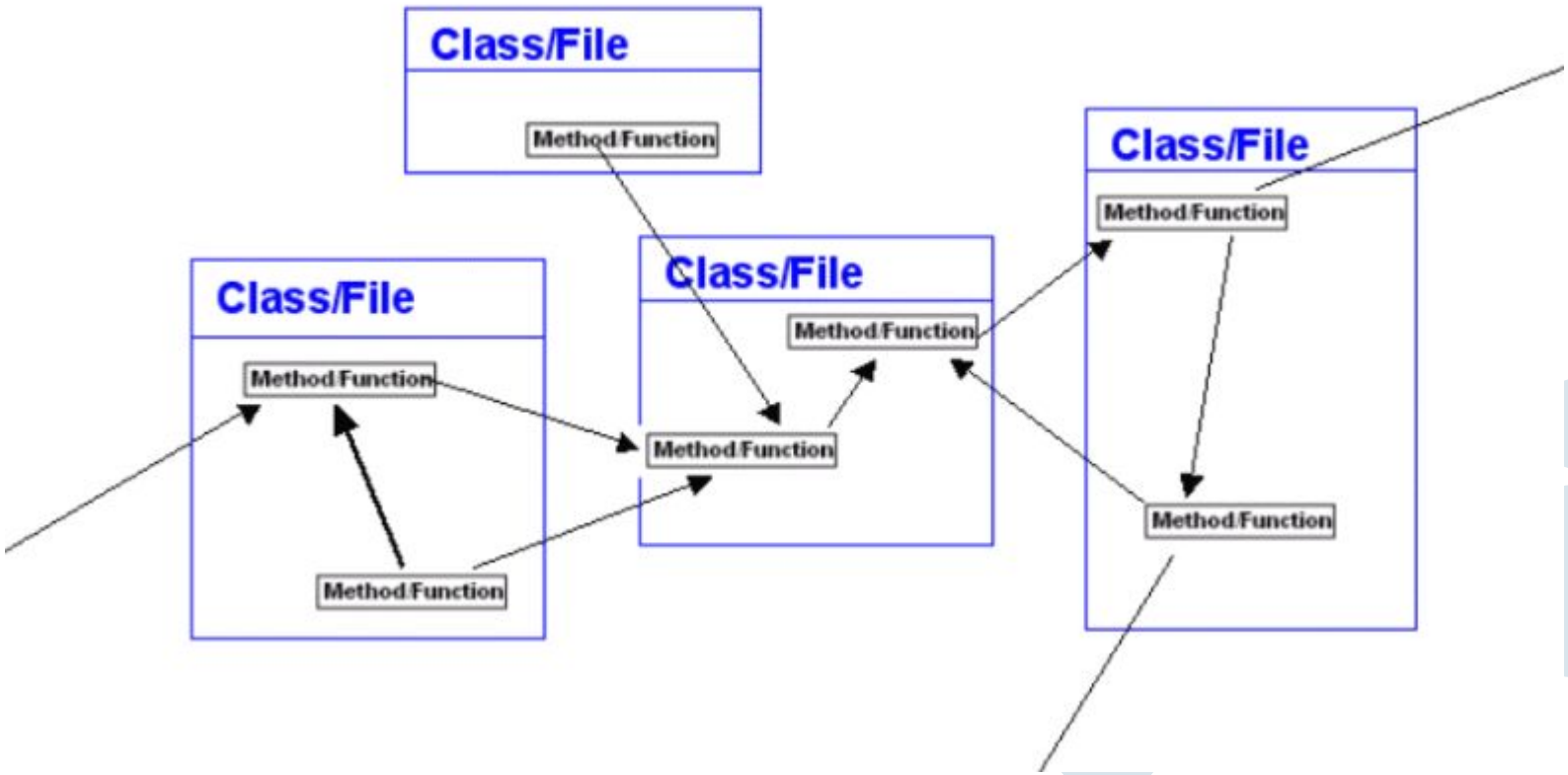
Assume a project with 100.000 Lines of Code (LoC).

Or about 10.000 methods or functions



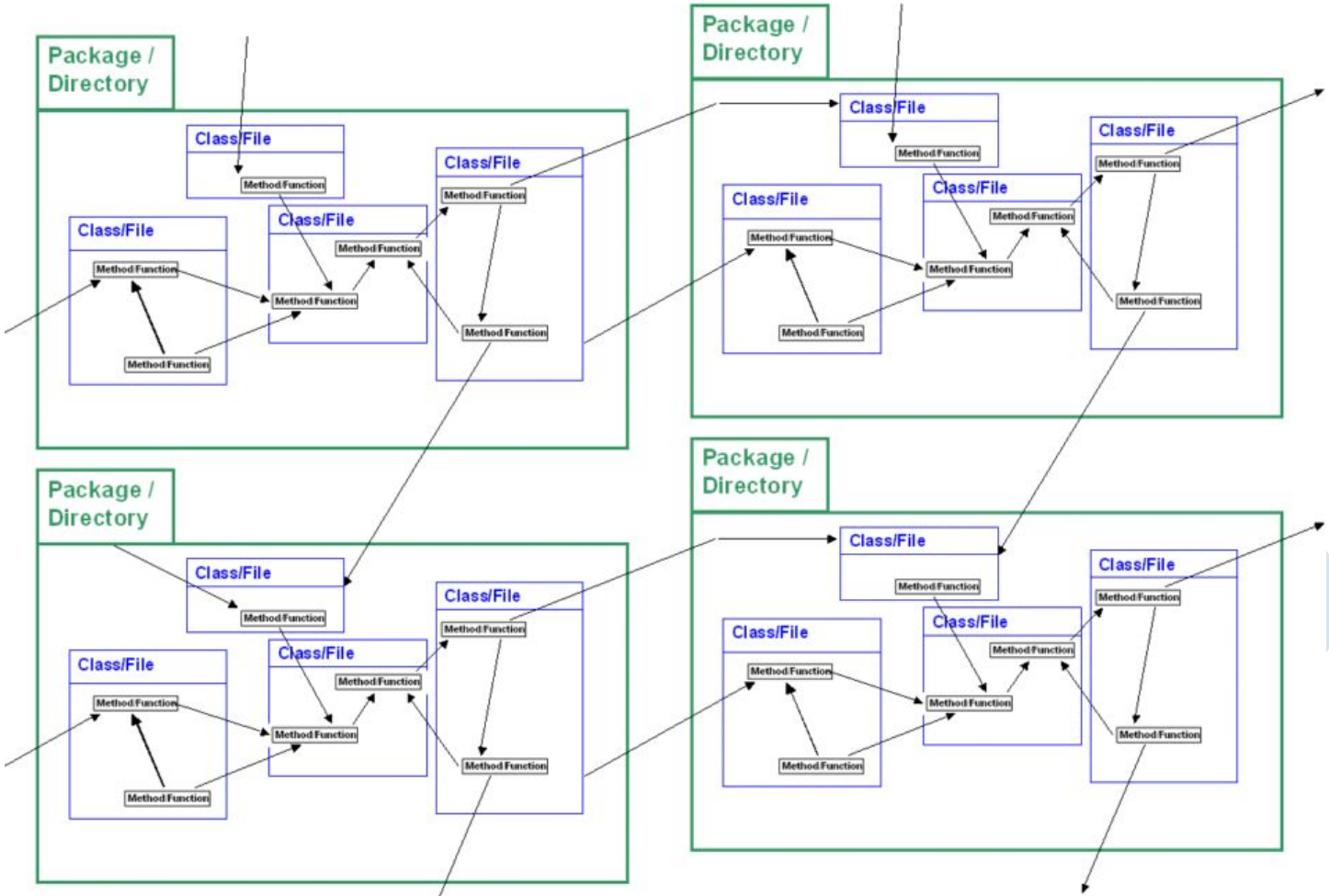
Analysis of Class Dependencies

Or 1000 classes.



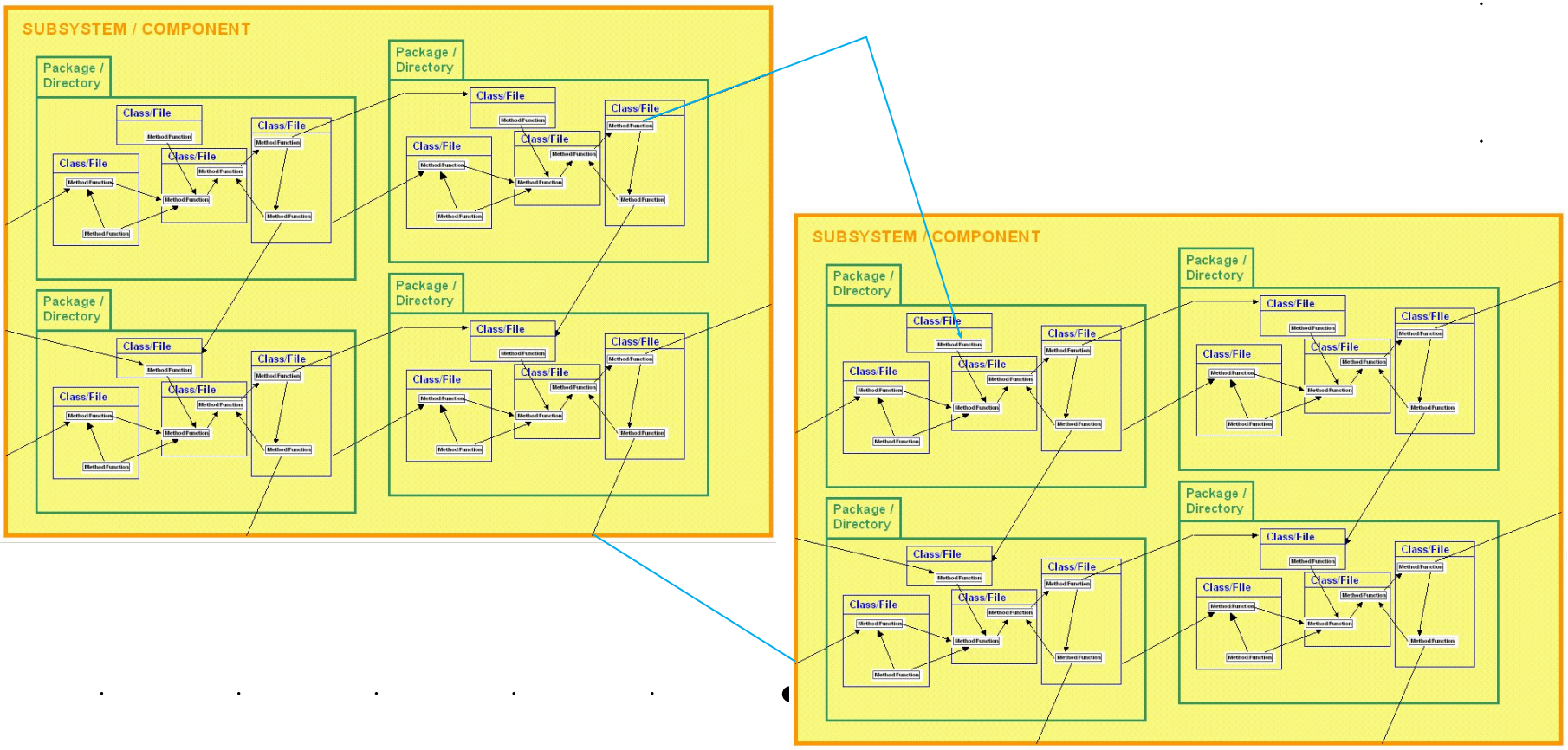
Analysis of Package Dependencies

Or 50 packages



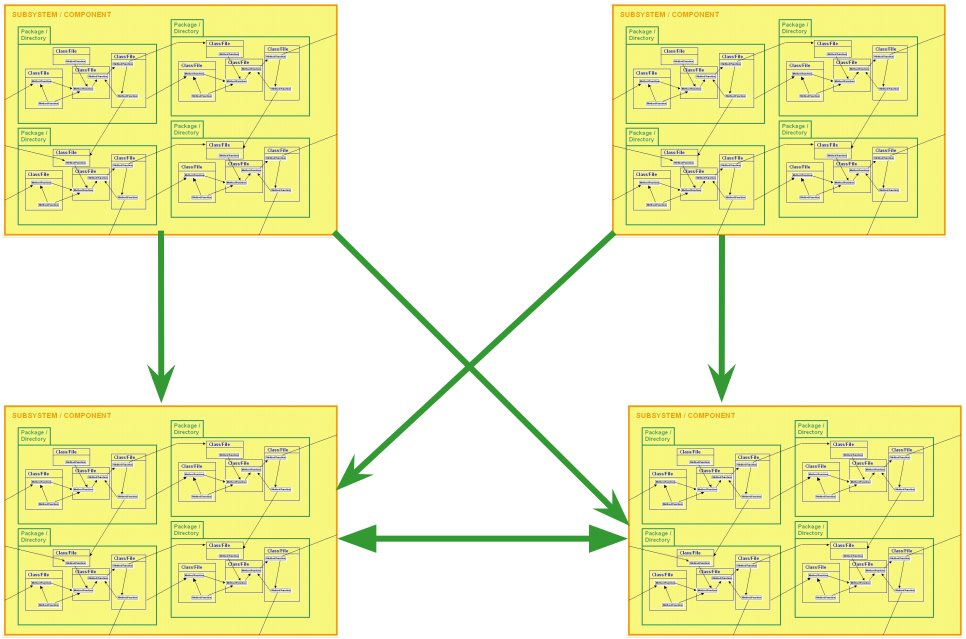
Analysis of Component Dependencies

Or 10 components.



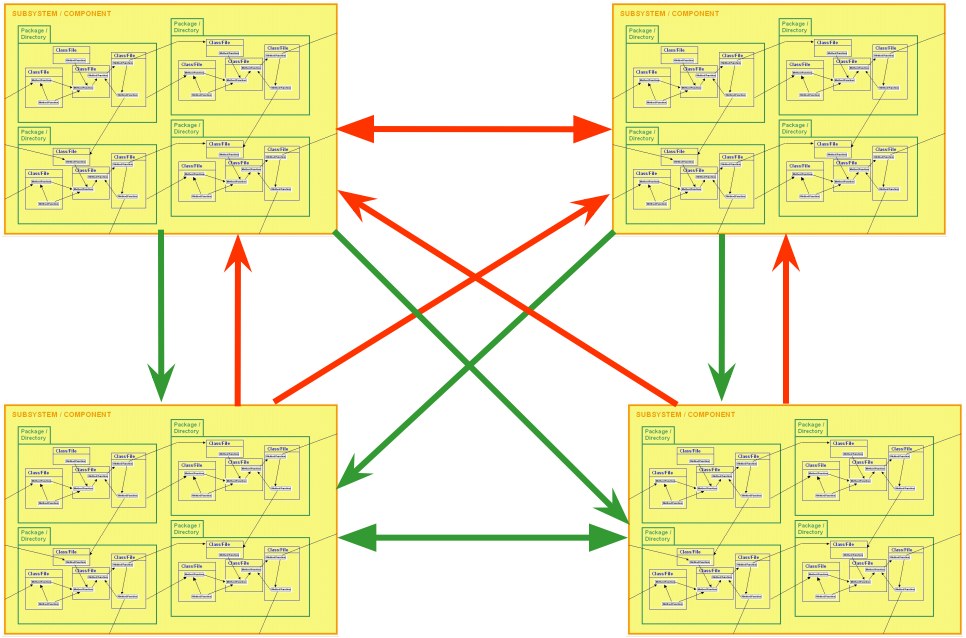
Analysis of Layers

Or 3 to 5 layers



Rules for Layers

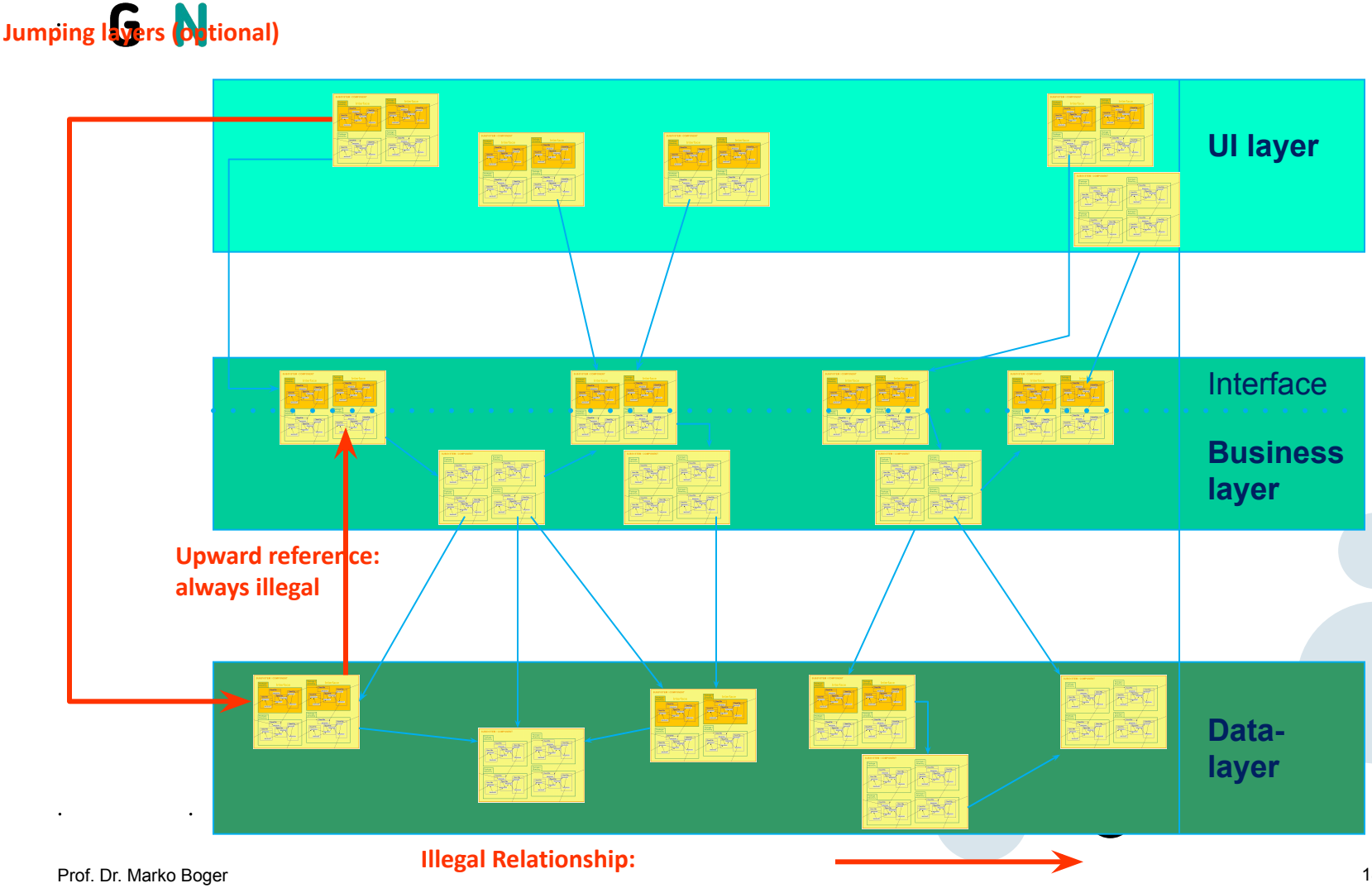
We only allow dependencies to go down, never up.



Allowed Relationships

Not allowed Relationships

Layered Architecture



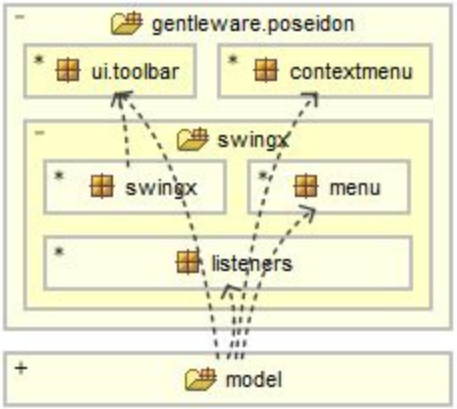
Tooling for Structure Analysis

Headway Structure 101 provides an Architecture Diagram.

It is automatically generated based on existing dependencies and package structure.

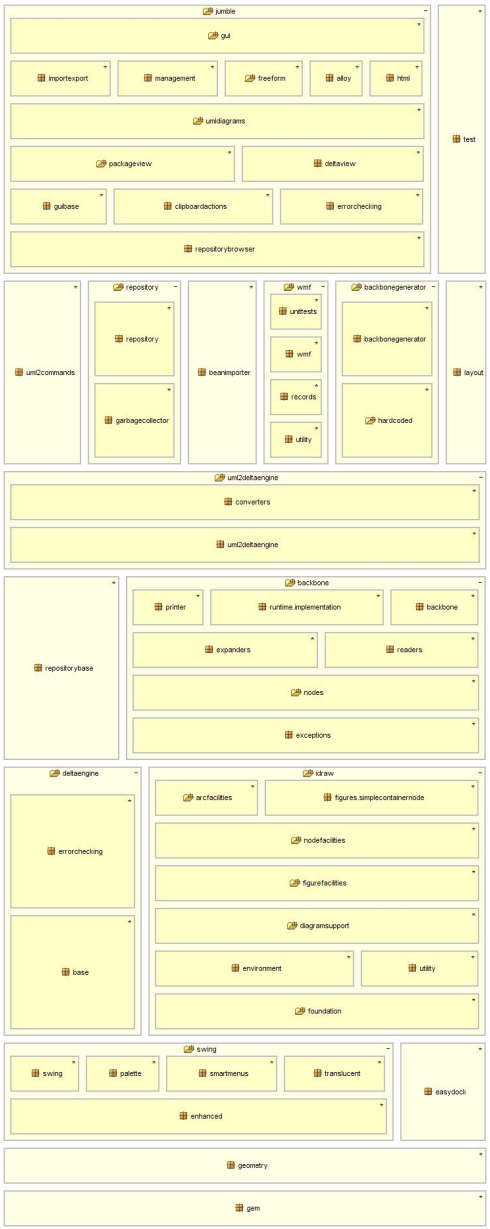
It can be thought of as a brick wall

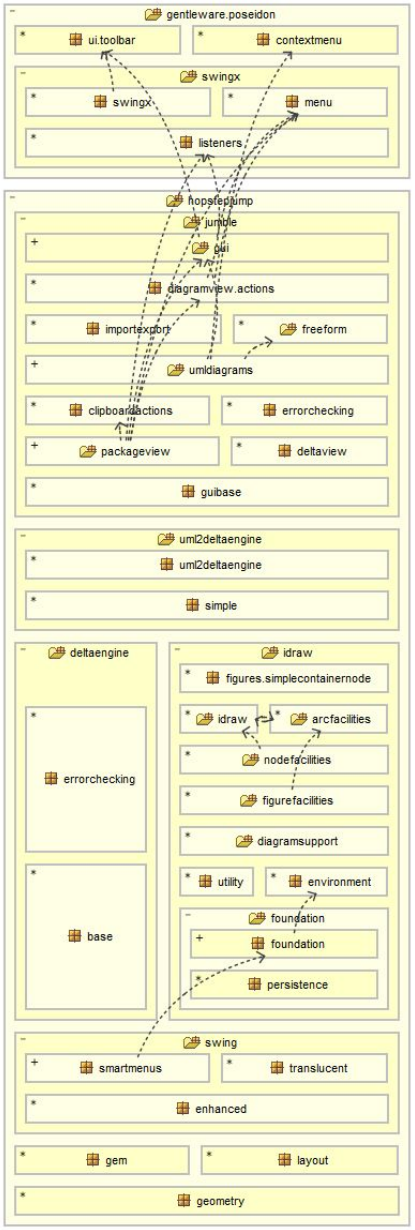
- Packages should only depend on blocks below it.
- Dependencies upward are violations and are shown



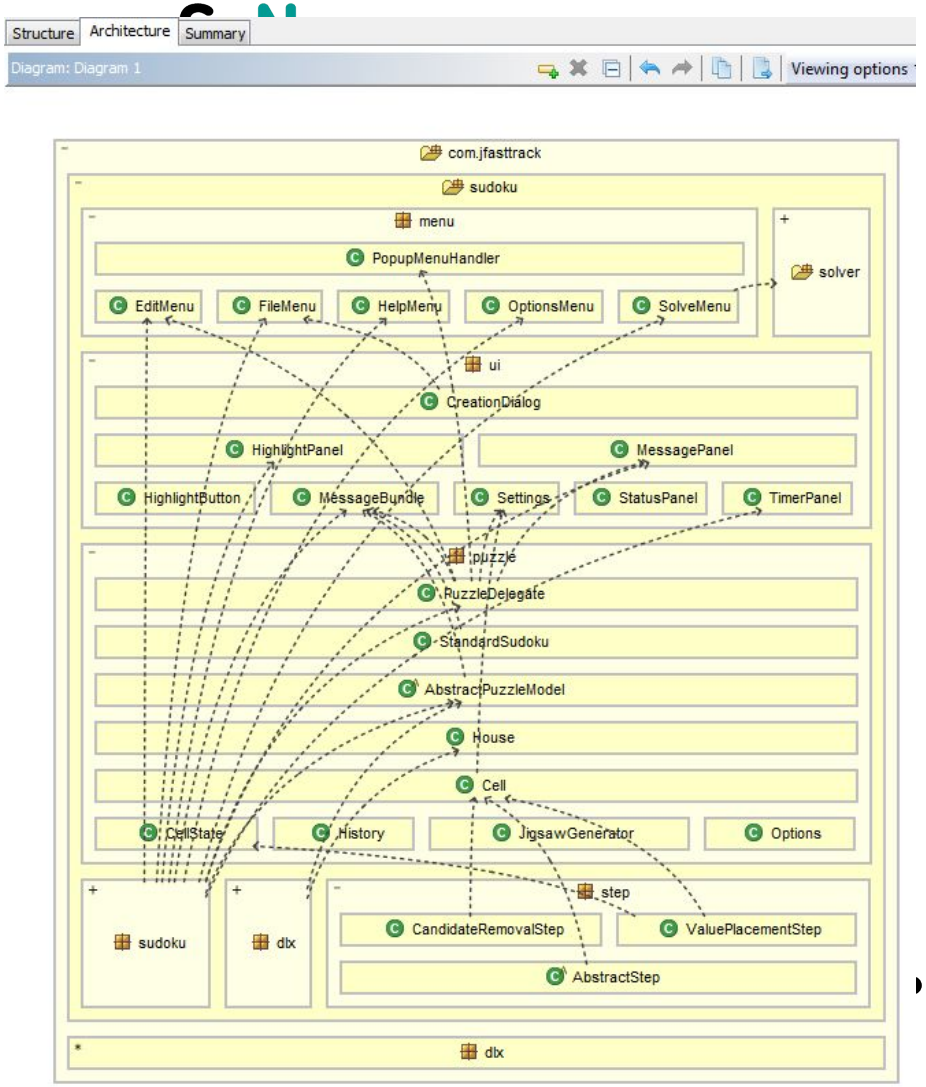
An Example

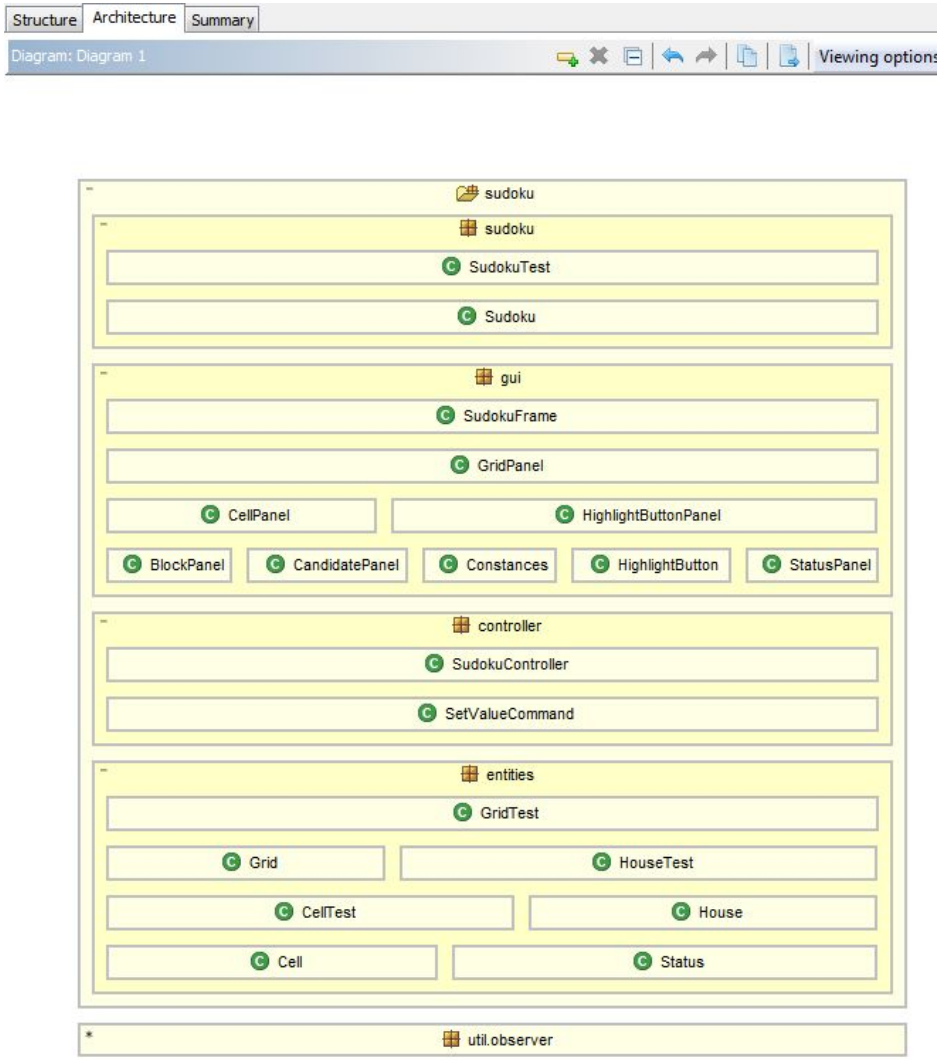
Structure Analysis of Poseidon at different times during development.





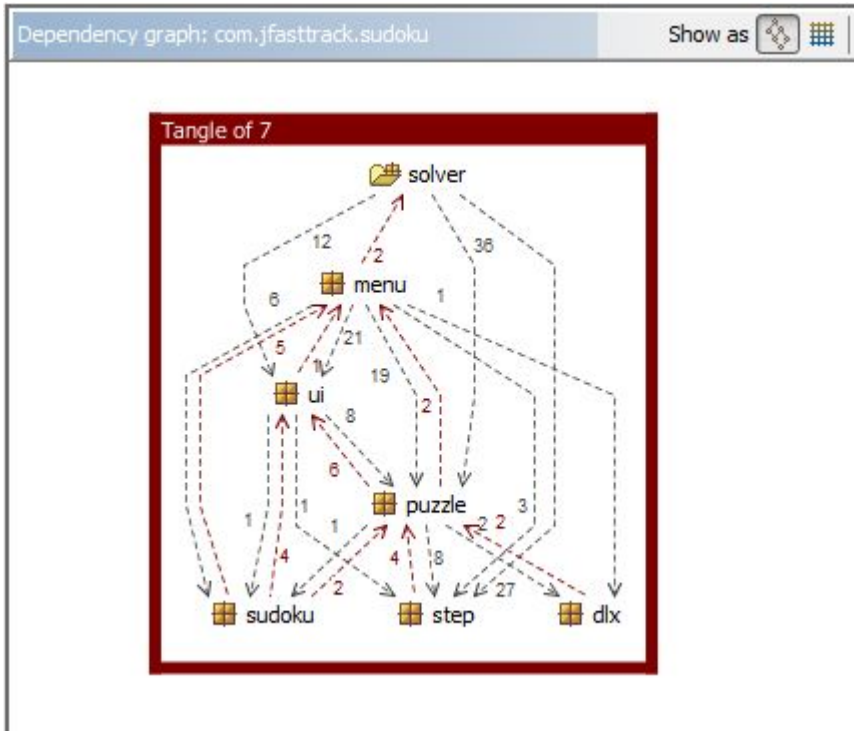
Example: Sudoku



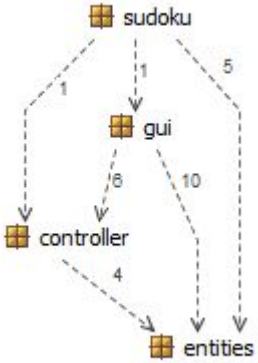


Dependency Graph in Structure 101

com.jfasttrack.sudoku



de.htwg.se.sudoku



Matrix View in Structure 101

com.jfasttrack.sudoku

Dependency graph: com.jfasttrack.sudoku

		Tangle of 7						
		menu	solver	ui	puzzle	sudoku	dlx	step
Tangle of 7	menu			1	2	5		
	solver	2						
	ui	21	12		6	4		
	puzzle	19	36	8		2	2	4
	sudoku	6		1	1			
	dlx	1			2			
	step	3	27	1	8			

de.htwg.se.sudoku

Dependency graph: de.htwg.sudoku Show as

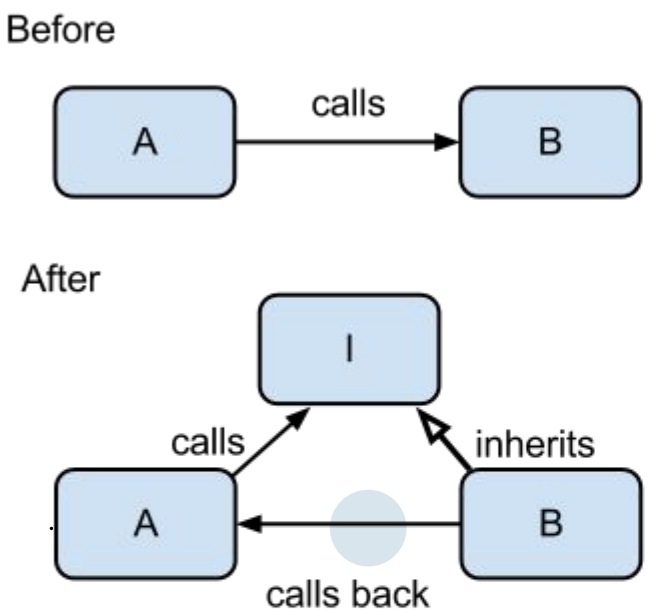
	sudoku	gui	controller	entities
sudoku				
gui	1			
controller	1	6		
entities	5	10	4	

How do we resolve Layer conflicts?

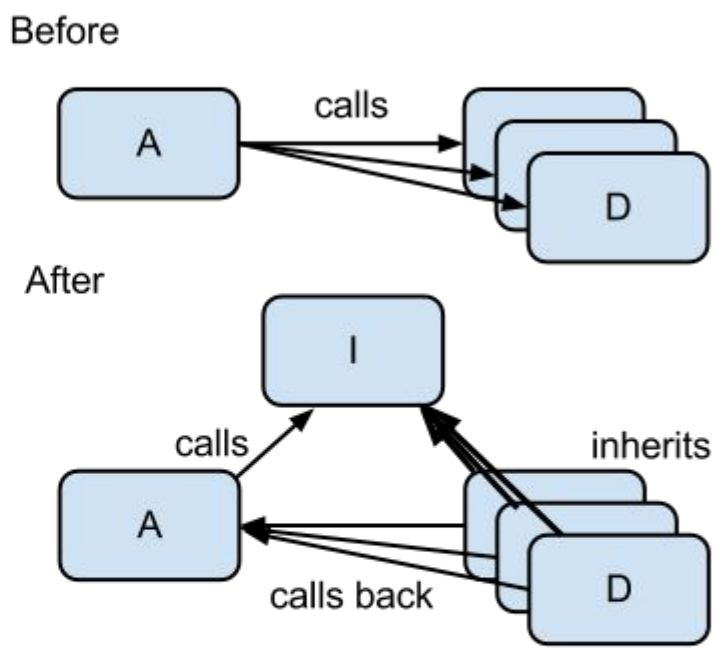
- Move classes, methods or variables
  - Move to a higher layer
- Move the functionality where it belongs
  - Do not jump layers
  - Do not make calls upwards
- Inversion of Control
  - Turn around the dependency
  - A typical implementation for IoC is the Observer Pattern

Inversion of Control

The call is inverted by introducing an Abstraction

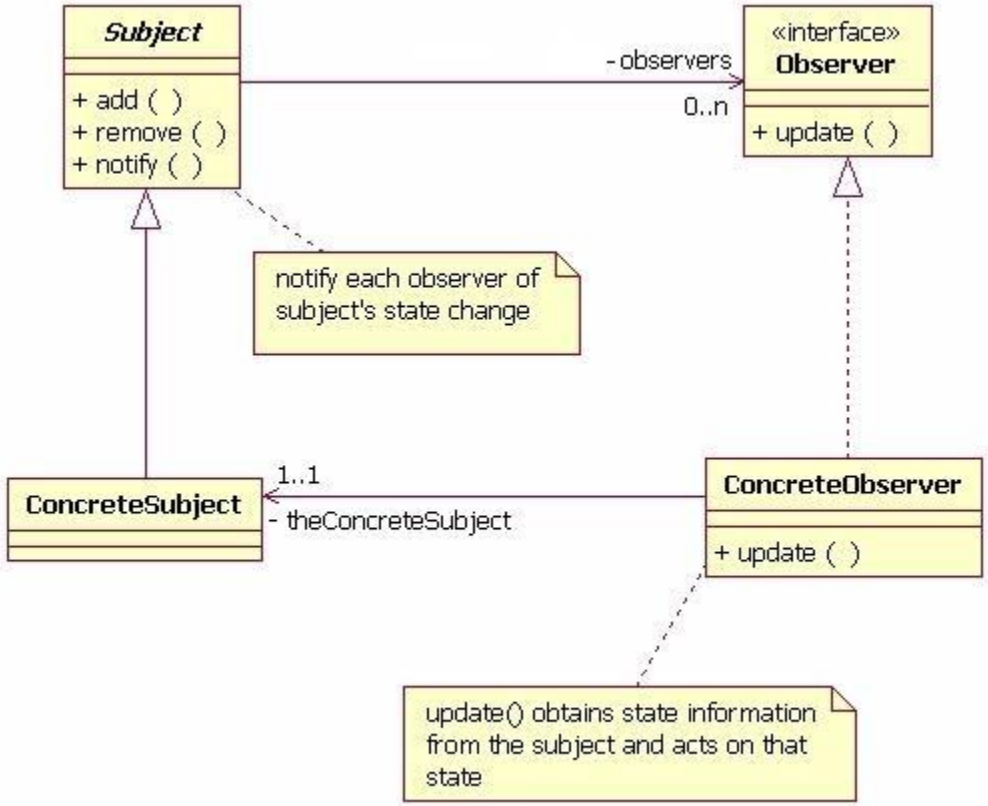


This is particularly important with several instances



Observer Pattern

A pattern that implements Inversion of Control is the Observer Pattern.



Implementation of Observer in Scala

- Observer
  - update
- Observable
  - add
  - remove
  - notify

```scala
package de.htwg.util

trait Observer {
  def update
}

class Observable {
  var subscribers:Vector[Observer] = Vector()
  def add(s:Observer) = subscribers=subscribers:+s
  def remove(s:Observer) = subscribers=subscribers.filterNot(o=>o==s)
  def notifyObservers = subscribers.foreach(o=>o.update)
}
```

Model View Controller or MVC

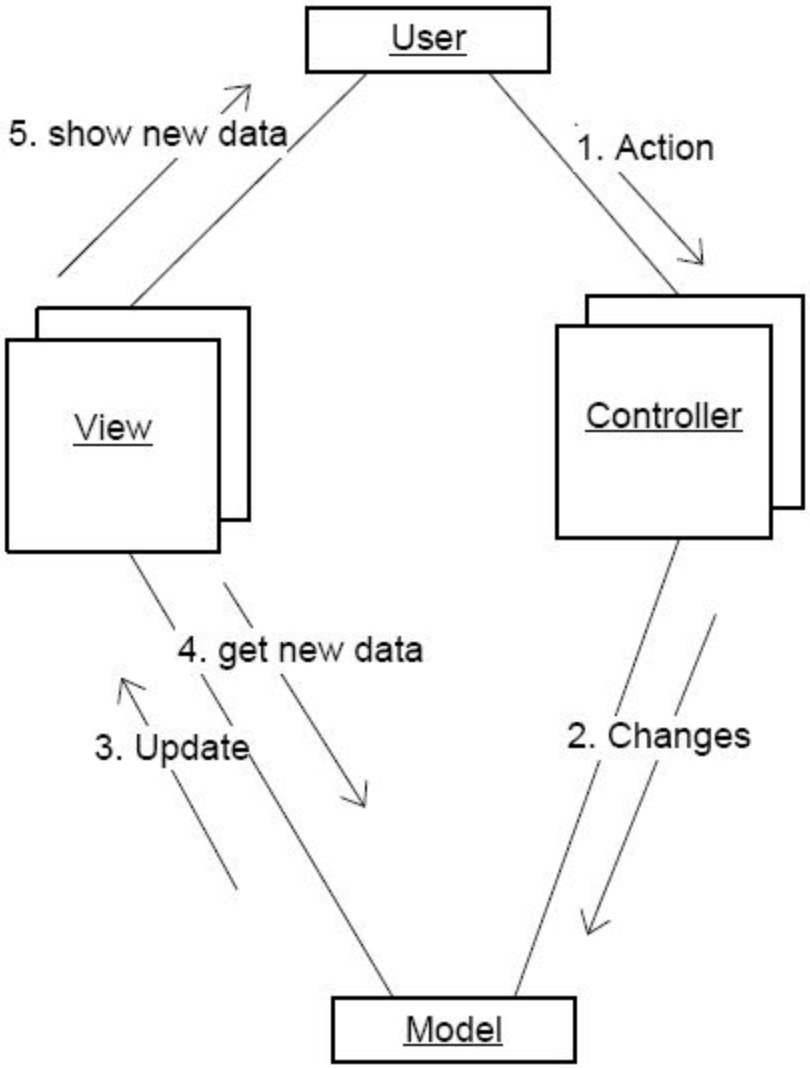
MVC is an architecture pattern.

It was developed in 1979 for Smalltalk by Trygve Reenskaug.

It is today the standard architecture pattern for most event driven software, like GUIs.

It also plays an important role for web-architectures.

We will use a strict version of MVC, where View, Controller and Model are layers.



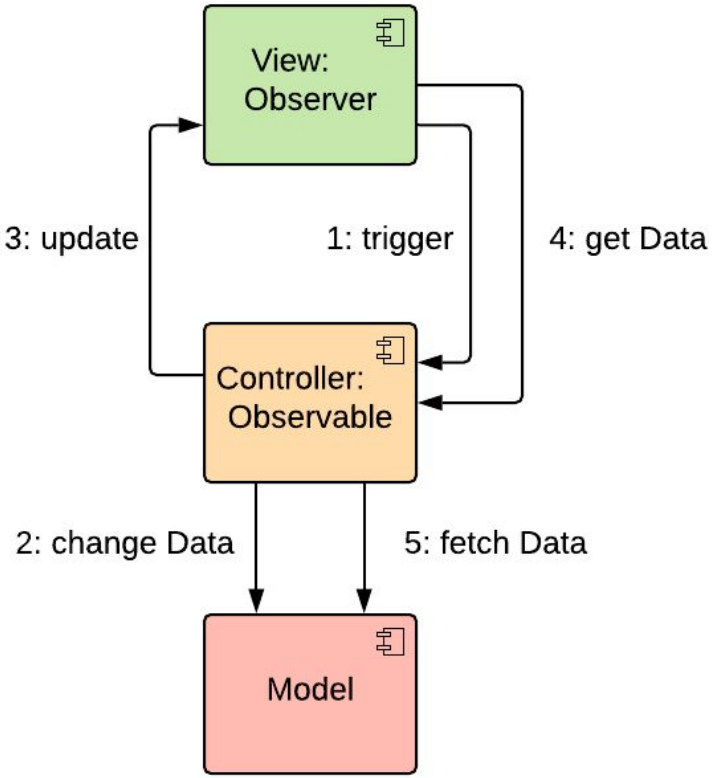
Model View Controller revised

To achieve strict layering, the Controller is the Observable.

The View receives interactions of the User, translates them into modality-independent actions and triggers the action on the Controller.

The Controller changes the Data in the Model and sends an update signal to the View.

The View gets the new Data from the Controller (which fetches it from the Model) and updates the view.



The Role of the Controller

The Controller contains Business logic.

It is independent of the modality.

A Modality is a channel for input or output between computer and human, like keyboard (TUI), mouse (GUI, WUI) or touch (mobile).

The View translates modality-dependant interactions into modality-independant actions.

The Controller supervises the business process, i.e. with the management of a state machine.

It contains only transient or no data. All persistent data and state is transferred to the Model.

The Role of the View

The View layer translates user input into controller calls.

It holds no business logic and no data. In later variants of this architecture the view can cache some data (Web tech).

The Role of the Model

The Model contains all data and the logic on this data.

However, it does not contain logic across the model, but only local logic.

All state should be in the model. The model will eventually be written to a database, only the model layer is saved. All layers above are stateless or have transient state.

Task 5: Build a MVC Architecture

- Make sure that your game has a clean MVC architecture
- No import of classes from higher layers
- Use reorganization and observer pattern to resolve
  - Cycles
  - Layer conflicts